Sommario

ITERAZIONE 2 2
1. Introduzione 2
1.1. Aggiornamento dei casi d'uso 2
2. Analisi 2
2.2. Caso d'uso UC4: ACQUISTO BIGLIETTO 3
2.2.1. Diagramma di Sequenza di Sistema 3
2.2.2. Contratti delle operazioni 4
2.3. Caso d'uso UC8: GESTIONE WISHLIST 5
2.3.1. Diagramma di Sequenza di Sistema 5
2.3.2. Contratti delle operazioni 5
2.4. Caso d'uso UC9: GESTIONE EVENTO POP-UP 6
2.4.1. Diagramma di Sequenza di Sistema 6
2.4.2. Contratti delle operazioni 7
3. Progettazione 7
3.1. Caso d'uso d'avviamento 8
3.2. Diagrammi di sequenza UC4 8
3.2.1. selezionaEvento() 8
3.2.2. acquistaBiglietto() 9
3.2.3. confermaAcquisto() 9
3.3. Diagrammi di sequenza UC8 10
3.3.1. aggiungiInWishlist() 10
3.3.2. rimuoviDaWishlist() 10
3.3.3. mostraWishlist() 10
3.4. Diagrammi di sequenza UC9 11
3.4.1. creaPopUp() 11
3.4.2. inviaInvitiPrioritari() 12
3.4. Diagramma delle Classi (DCD) 12
4. Testing 13

ITERAZIONE 2

1. Introduzione

Nel corso di questa seconda iterazione, verrà posta l'attenzione nei confronti dell'implementazione di nuovi casi d'uso e dell'utilizzo di ulteriori pattern GoF, utili per raffinare il modello di dominio.

In particolare, verranno implementati gli scenari di successo dei seguenti casi d'uso:

- UC4: AQUISTO BIGLIETTO
- UC8: GESTIONE WISHLIST
- UC9: GESTIONE EVENTO POP-UP

1.1. Aggiornamento dei casi d'uso

Come per la prima iterazione, durante questa fase, sono stati rivisti alcuni dettagli dei casi d'uso implementati. Per visionare le suddette modifiche, è possibile consultare il documento "Modello dei casi d'uso" riportato nella cartella. Sono state inoltre raffinate le regole di dominio del sistema.

2. Analisi

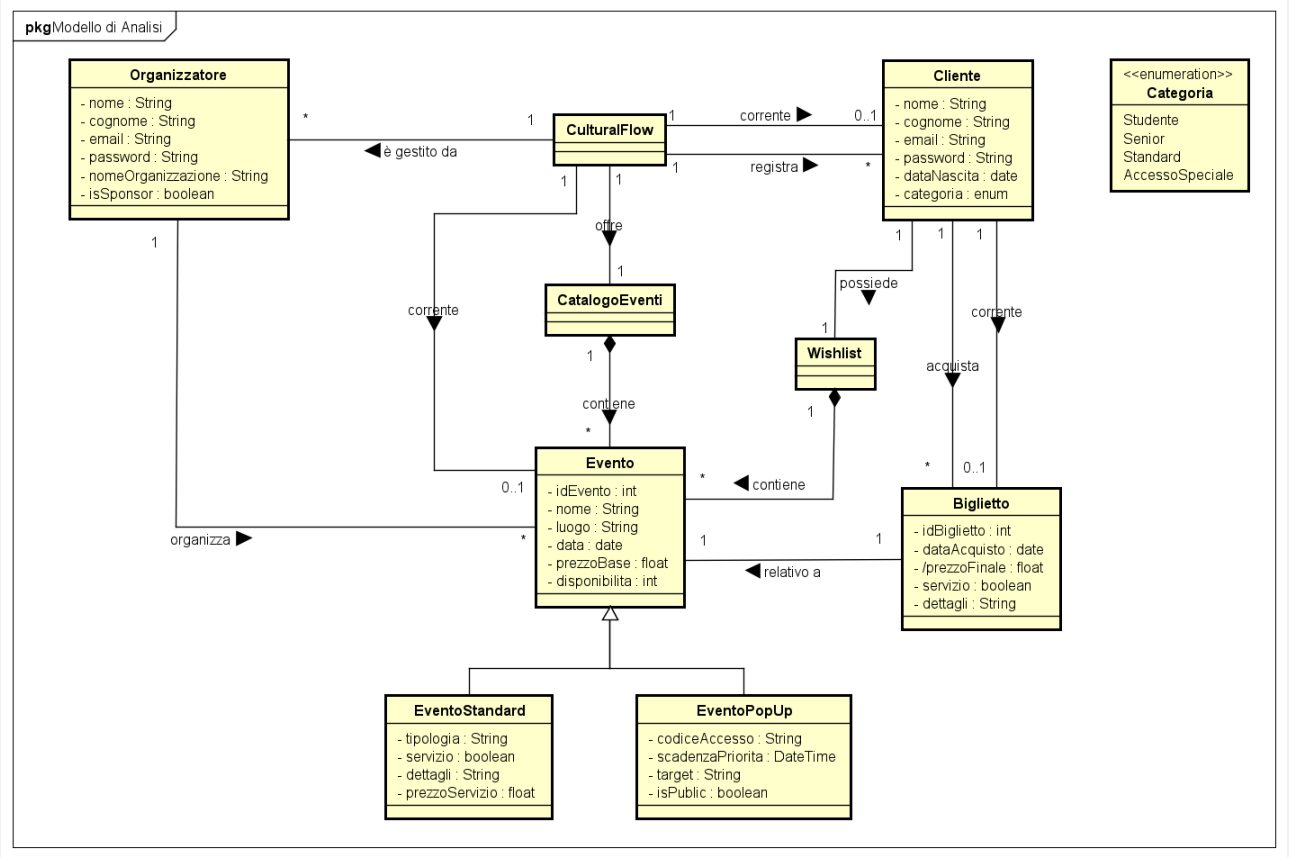
Nella fase di Analisi della seconda iterazione, sono state introdotte due nuove classi concettuali per permettere l'introduzione di alcuni casi d'uso. È stata, inoltre, modellata una generalizzazione riguardante i tipi di Evento. Sono state aggiunte le seguenti classi:

- Wishlist: rappresenta la wishlist di un Cliente;
- Biglietto: rappresenta un biglietto acquistato da un Cliente, con tutti i parametri necessari per il calcolo del prezzo;
- EventoPopUp: rappresenta un evento Pop-Up, indirizzato a una fascia ristretta di clienti;
- EventoStandard: rappresenta un evento base gestito dal Sistema.

Sono stati aggiornati anche alcuni parametri di alcune classi concettuali già definite in precedenza:

- È stato aggiunto il parametro disponibilita alla classe Evento, per permettere di aggiornare il numero di posti disponibili per quel determinato evento;
- È stato aggiunto l'attributo isSponsor alla classe Organizzatore per consentire agli Sponsor di creare eventi Pop-Up.
- Alcuni degli attributi precedentemente inseriti nella classe Evento, sono stati adesso spostati nella sottoclasse EventoStandard.

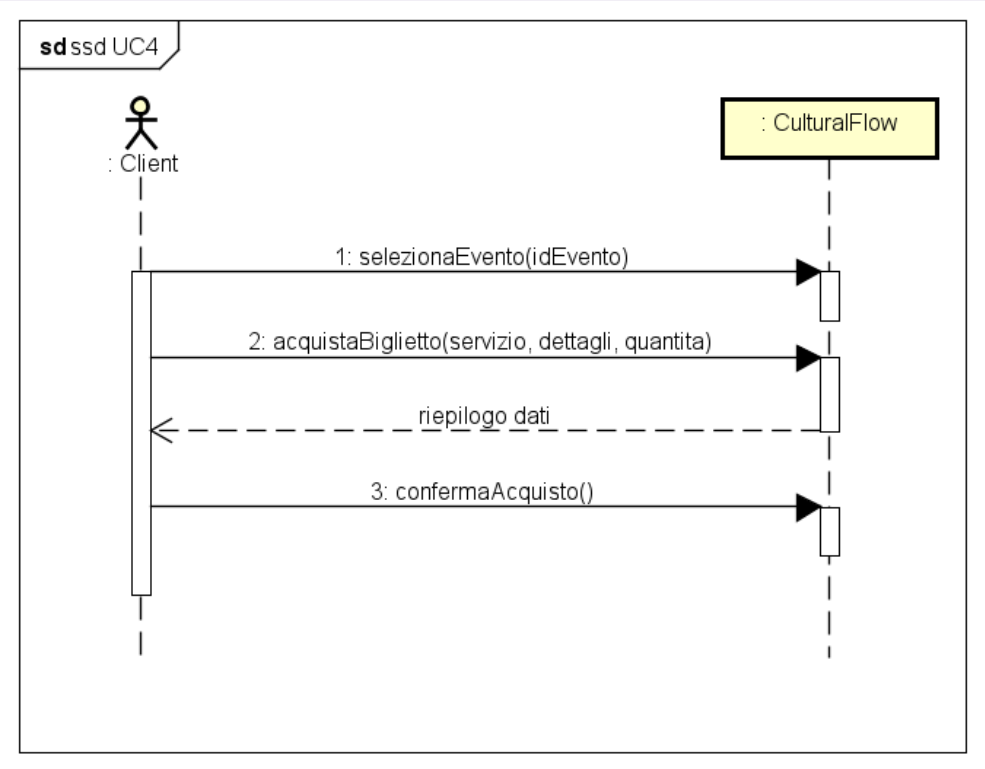
Si riporta di seguito il Modello di Dominio aggiornato:



2.2. Caso d'uso UC4: ACQUISTO BIGLIETTO

2.2.1. Diagramma di Sequenza di Sistema

Il seguente SSD mostra l'iterazione tra il Cliente e il Sistema al fine di effettuare l'acquisto di uno o più biglietti.



2.2.2. Contratti delle operazioni

I seguenti contratti descrivono le principali operazioni sulla base dell'SSD precedentemente mostrato.

CONTRATTO 1, UC4

Operazione:	selezionaEvento(idEvento:int)
Riferimenti:	Caso d'uso: Acquisto biglietto
Pre-condizioni:	-
Post-condizioni:	- è stata recuperata un'istanza e di Evento attraverso l'attributo idEvento.

CONTRATTO 2, UC4

Operazione:	acquistaBiglietto(servizio:String, dettagli:String, quantita:int)
Riferimenti:	Caso d'uso: Acquisto biglietto
Pre-condizioni:	- è stato selezionato un Evento e
Post-condizioni:	- sono state create n istanze di Biglietto (n = quantita); - sono stati inizializzati gli attributi dei Biglietti; - viene aggiornato l'attributo disponibilita di Evento - l'istanza b di Biglietto viene associata a Cliente tramite l'associazione "corrente"; - l'istanza b di Biglietto viene associata all'istanza e di Evento attraverso l'associazione "relativo a".

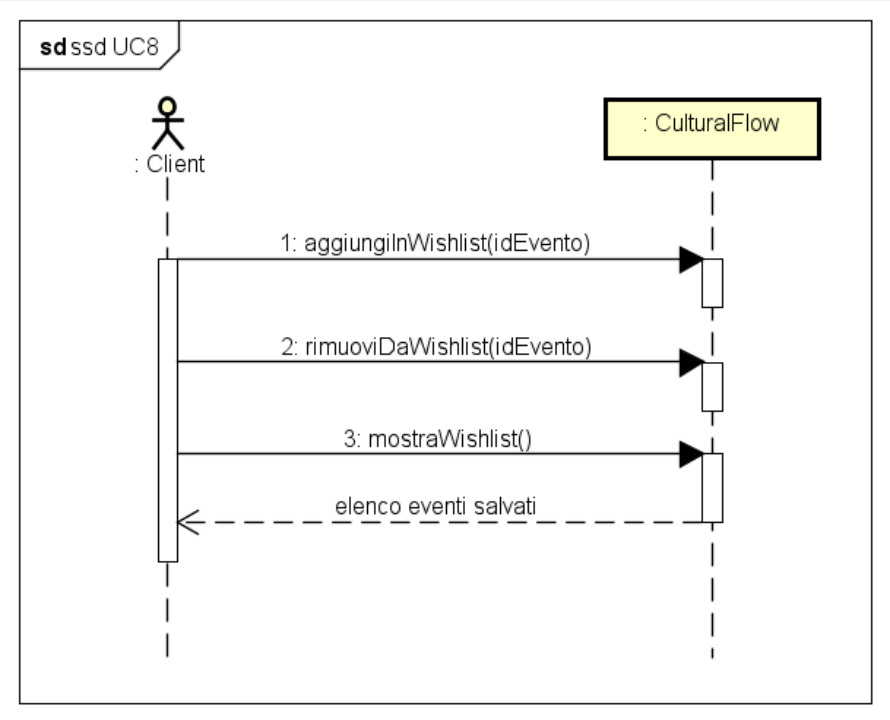
CONTRATTO 3, UC4

Operazione:	confermaAcquisto()
Riferimenti:	Caso d'uso: Acquisto biglietto
Pre-condizioni:	- sono state create n istanze di Biglietto
Post-condizioni:	- ogni istanza b di Biglietto viene associata a Cliente tramite l'associazione "acquista".

2.3. Caso d'uso UC8: GESTIONE WISHLIST

2.3.1. Diagramma di Sequenza di Sistema

Il seguente diagramma di sequenza di sistema mostra l'interazione tra il Cliente e il Sistema con l'obiettivo da parte del primo di inserire, rimuovere e/o visualizzare la propria wishlist.



2.3.2. Contratti delle operazioni

I seguenti contratti descrivono le principali operazioni sulla base dell'SSD precedentemente mostrato.

CONTRATTO 1, UC8

Operazione:	aggiungiInWishlist(idEvento:int)
Riferimenti:	Caso d'uso: Gestione Wishlist
Pre-condizioni:	-
Post-condizioni:	- è stata recuperata un'istanza e di Evento; - l'istanza e è stata associata alla Wishlist tramite l'associazione "contiene".

CONTRATTO 2, UC8

Operazione:	rimuoviDaWishlist(idEvento:int)
Riferimenti:	Caso d'uso: Gestione Wishlist
Pre-condizioni:	-
Post-condizioni:	- è stata recuperata un'istanza e di Evento; - l'istanza e è stata rimossa dalla Wishlist.

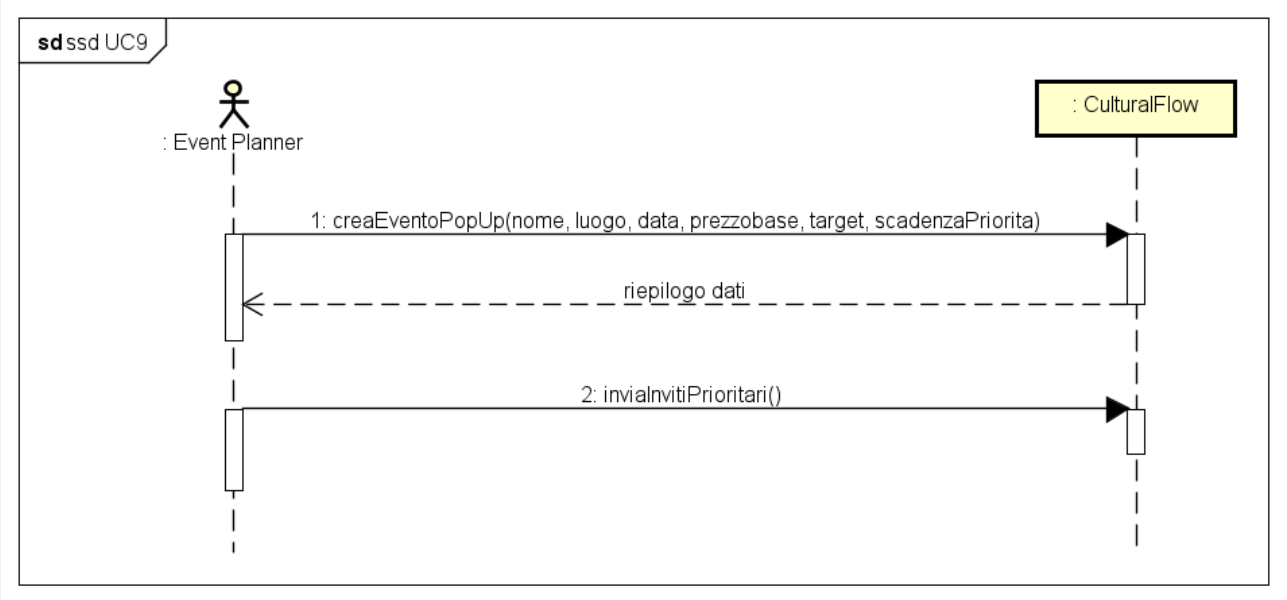
CONTRATTO 3, UC8

Operazione:	mostraWishlist()
Riferimenti:	Caso d'uso: Gestione Wishlist
Pre-condizioni:	-
Post-condizioni:	- sono state recuperate le istanze e di Evento contenute in Wishlist.

2.4. Caso d'uso UC9: GESTIONE EVENTO POP-UP

2.4.1. Diagramma di Sequenza di Sistema

Il seguente SSD mostra l'interazione tra un Organizzatore/Sponsor e il Sistema al fine di inserire un Evento Pop-Up e di inviare le e-mail con il codice d'accesso prioritario a chiunque abbia mostrato interessi simili a quelli proposti dall'evento.



2.4.2. Contratti delle operazioni

I seguenti contratti descrivono le principali operazioni sulla base dell'SSD precedentemente mostrato.

CONTRATTO 1, UC9	
Operazione:	creaEventoPopUp(nome:String, luogo:String, data:date, prezzoBase:float, target:String, scadenzaPriorita:dateTime)
Riferimenti:	Caso d'uso: Gestione Evento Pop-Up e inviti da Wishlist
Pre-condizioni:	-
Post-condizioni:	- è stata creata un'istanza epu di EventoPopUp; - sono stati inizializzati gli attributi di epu; - l'istanza epu è stata associata a CulturalFlow tramite l'associazione "corrente".

CONTRATTO 2, UC9	
Operazione:	inviaInvitiPrioritari()
Riferimenti:	Caso d'uso: Gestione Evento Pop-Up e inviti da Wishlist
Pre-condizioni:	-
Post-condizioni:	- sono state recuperate le istanze c di Cliente in relazione alla propria Wishlist; - per ogni istanza c di Cliente viene inviata un'e-mail con il codiceAccesso; - l'istanza epu è stata associata a CatalogoEventi tramite l'associazione "contiene"; - l'istanza epu è stata associata a Organizzatore tramite l'associazione "organizza".

3. Progettazione

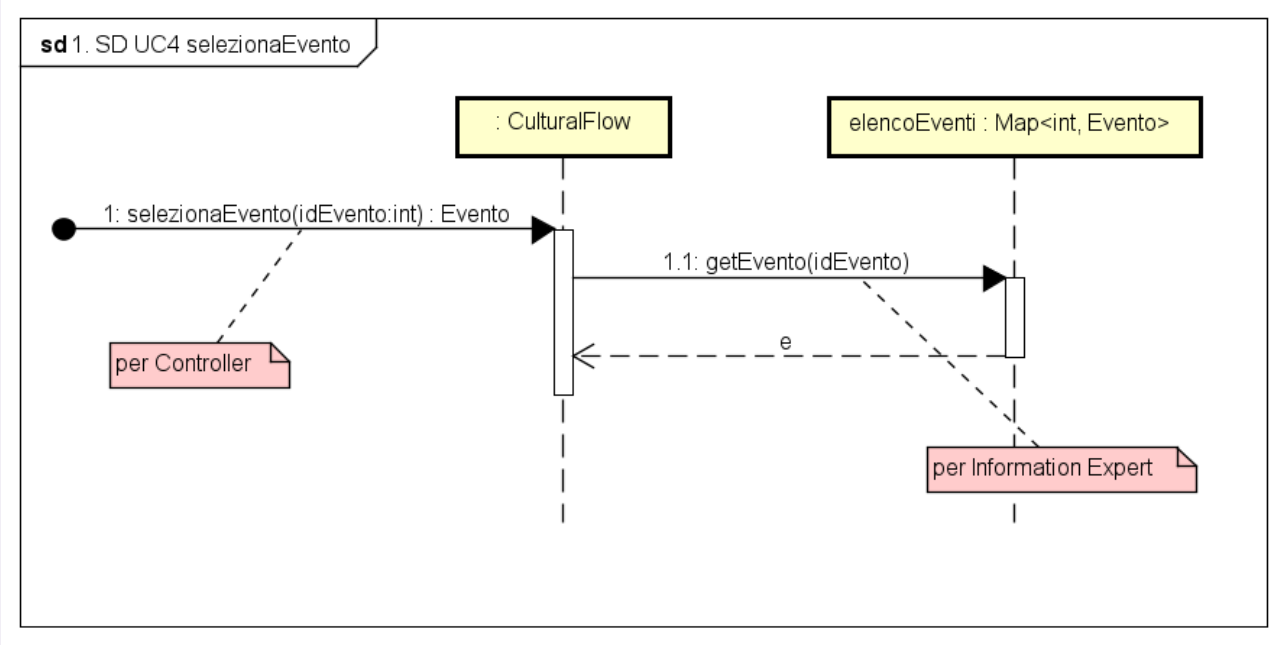
Durante questa fase, è stato aggiornato il DCD già precedentemente elaborato al fine di riflettere l'evoluzione del sistema. È stata, inoltre, posta l'attenzione verso l'implementazione di alcune delle regole di dominio, in particolare quelle relative alle tariffe agevolate per le varie categorie di Clienti. Per garantire flessibilità ed efficienza è stato, quindi, applicato il *pattern Strategy*, che ha permesso di incapsulare le diverse logiche di sconto e di applicarle relativamente al Cliente.

3.1. Caso d'uso d'avviamento

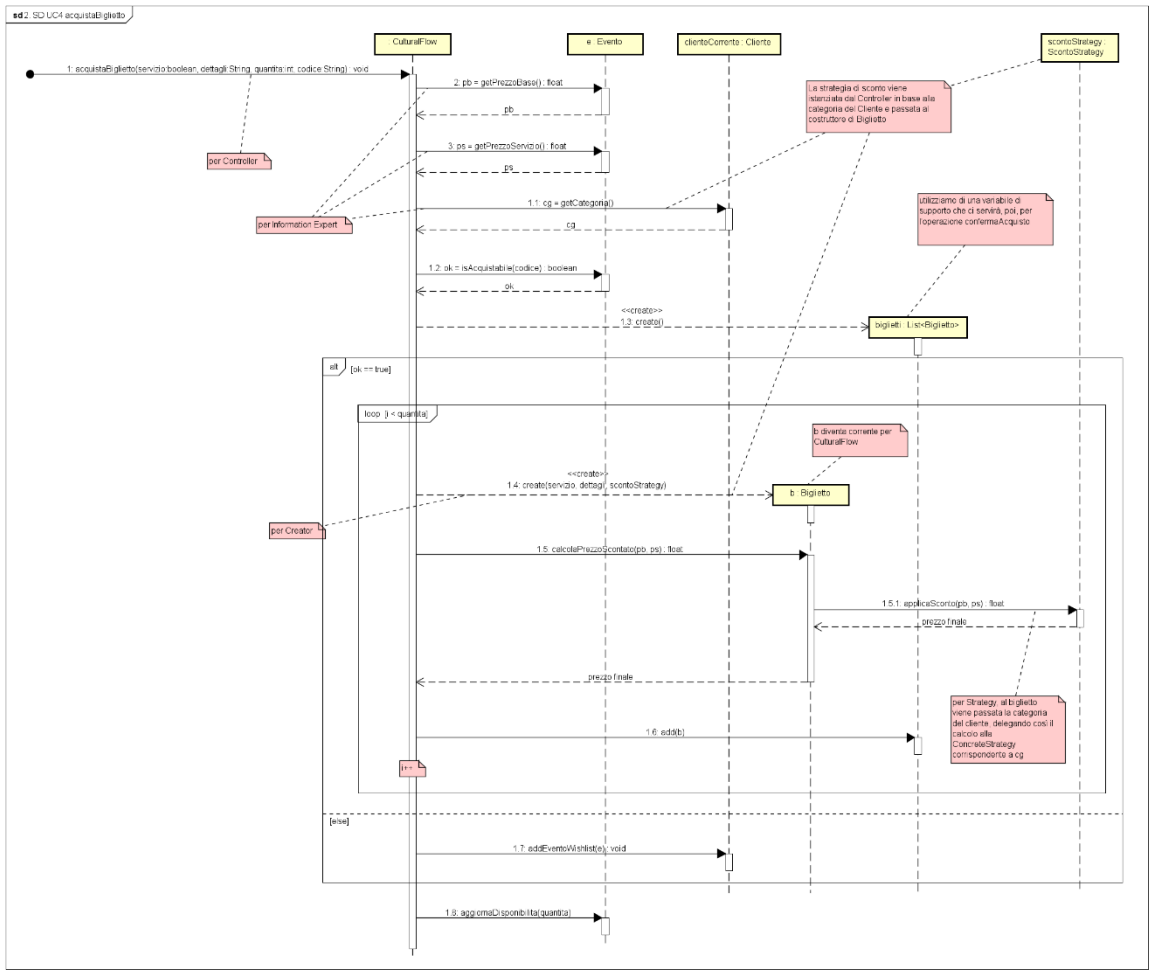
3.2. Diagrammi di sequenza UC4

In seguito, verranno riportati i diagrammi di sequenza relativi al caso d'uso UC4. È possibile notare l'applicazione di diversi pattern, sia GRASP che GoF: oltre il *pattern Controller*, già introdotto nell'iterazione 1, è stato applicato il *pattern Information Expert* per l'assegnazione delle responsabilità. Degno di nota è il *pattern Strategy*, che nel diagramma di sequenza acquistaBiglietto ha permesso la gestione di sconti differenti per le diverse categorie di Clienti, garantendo un sistema flessibile e facilmente estensibile.

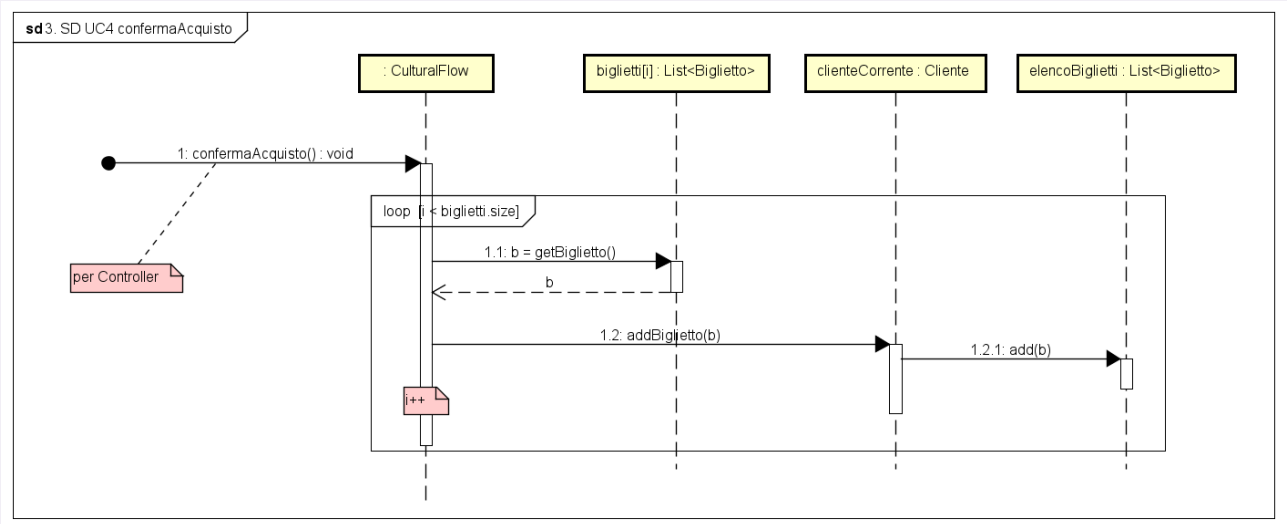
3.2.1. selezionaEvento()



3.2.2. acquistaBiglietto()

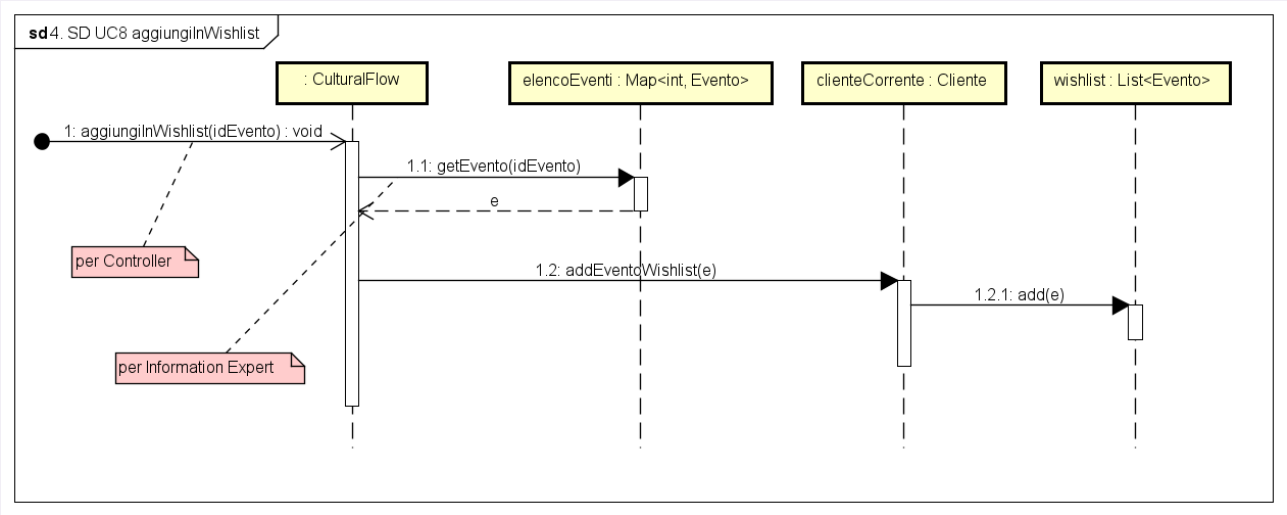


3.2.3. confermaAcquisto()

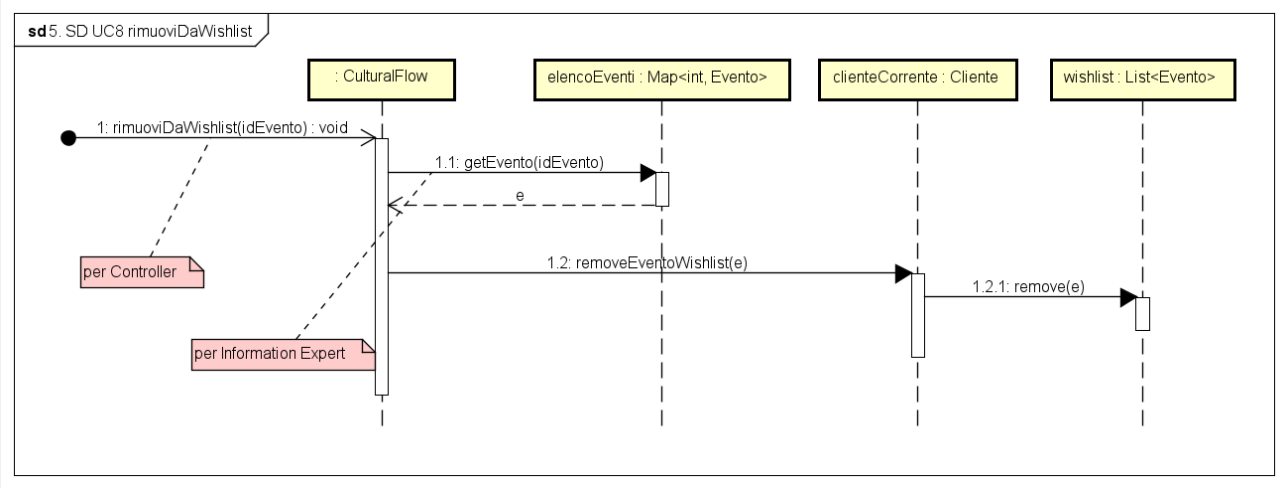


3.3. Diagrammi di sequenza UC8

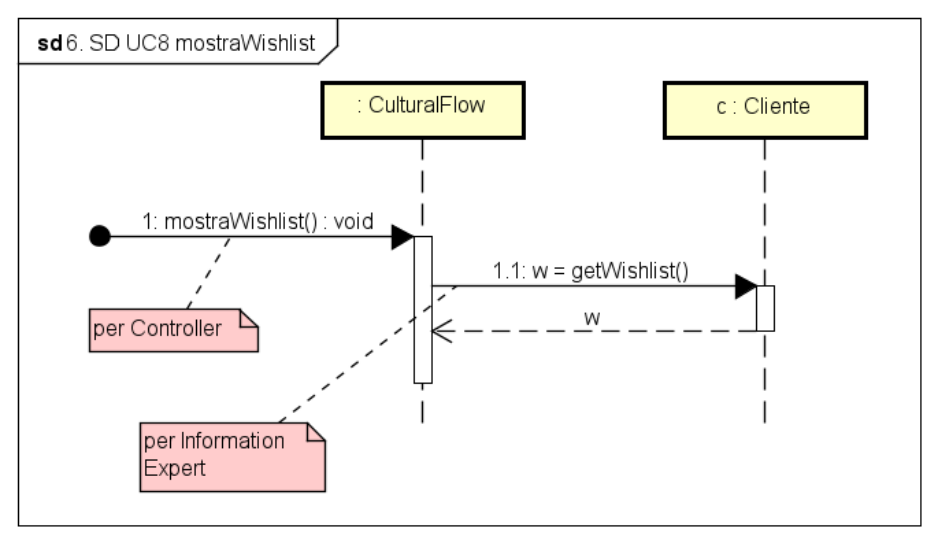
3.3.1. aggiungiInWishlist()



3.3.2. rimuoviDaWishlist()



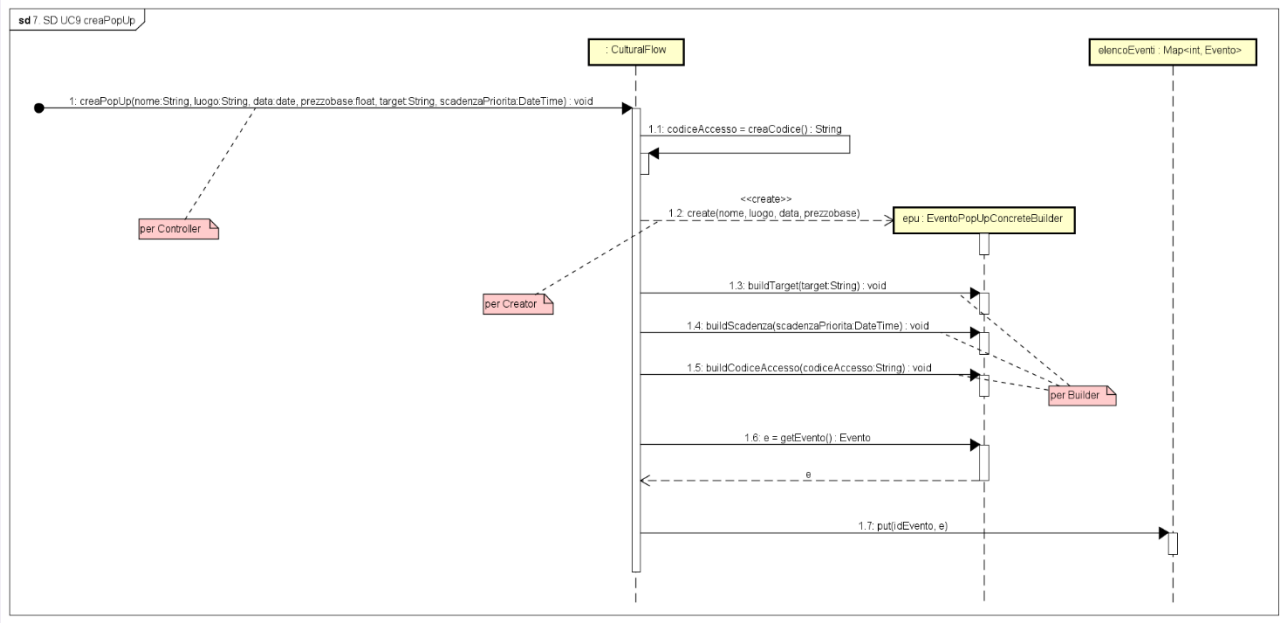
3.3.3. mostraWishlist()



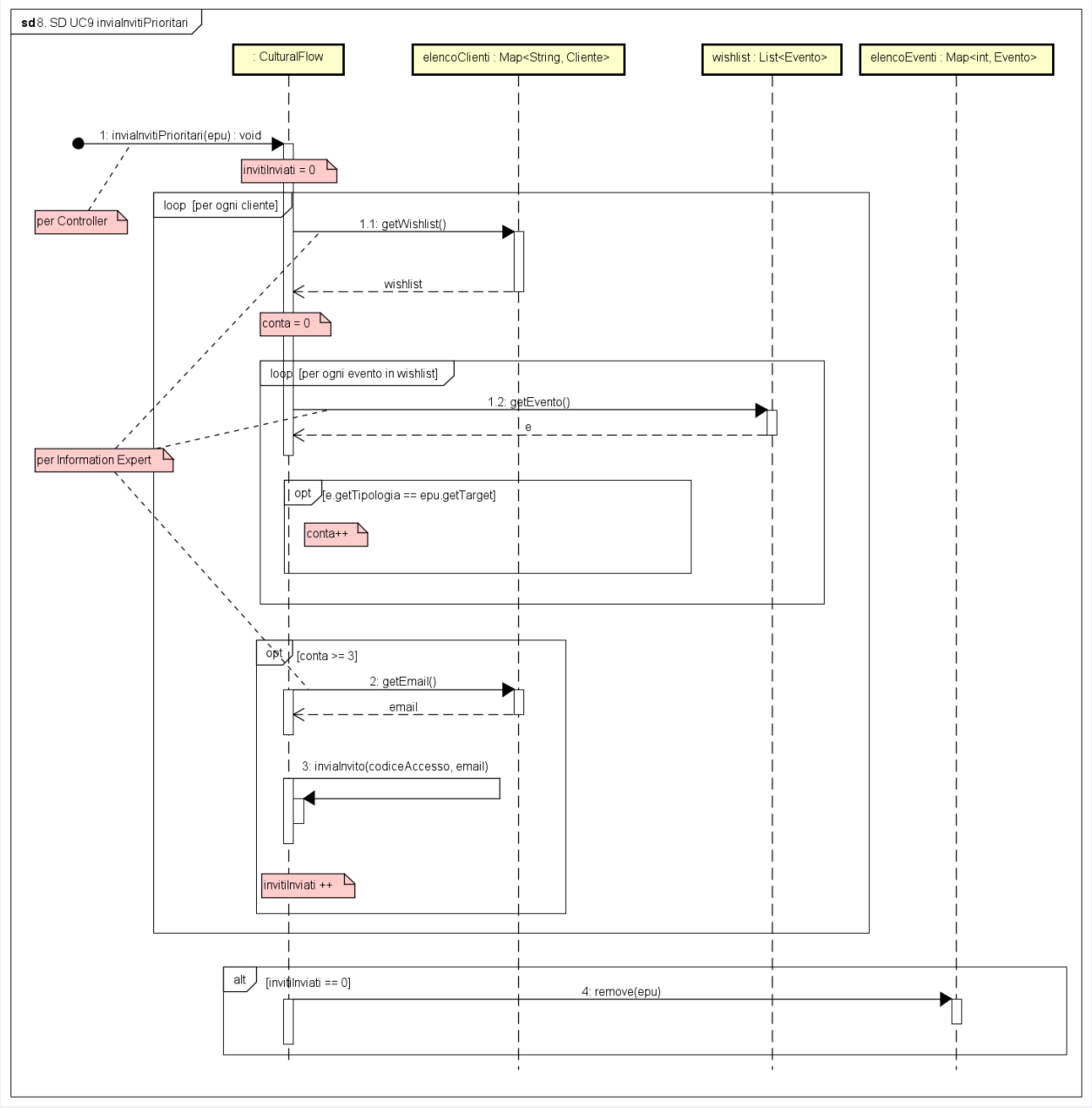
3.4. Diagrammi di sequenza UC9

Anche in questo caso d'uso, così come avevamo visto nell'iterazione 1 con il caso d'uso UC2, è stato applicato il *pattern creazionale Builder*. Qui l'obiettivo è quello di creare un Evento Pop-Up, consentendo di definire le sue caratteristiche in modo modulare e flessibile.

3.4.1. creaPopUp()

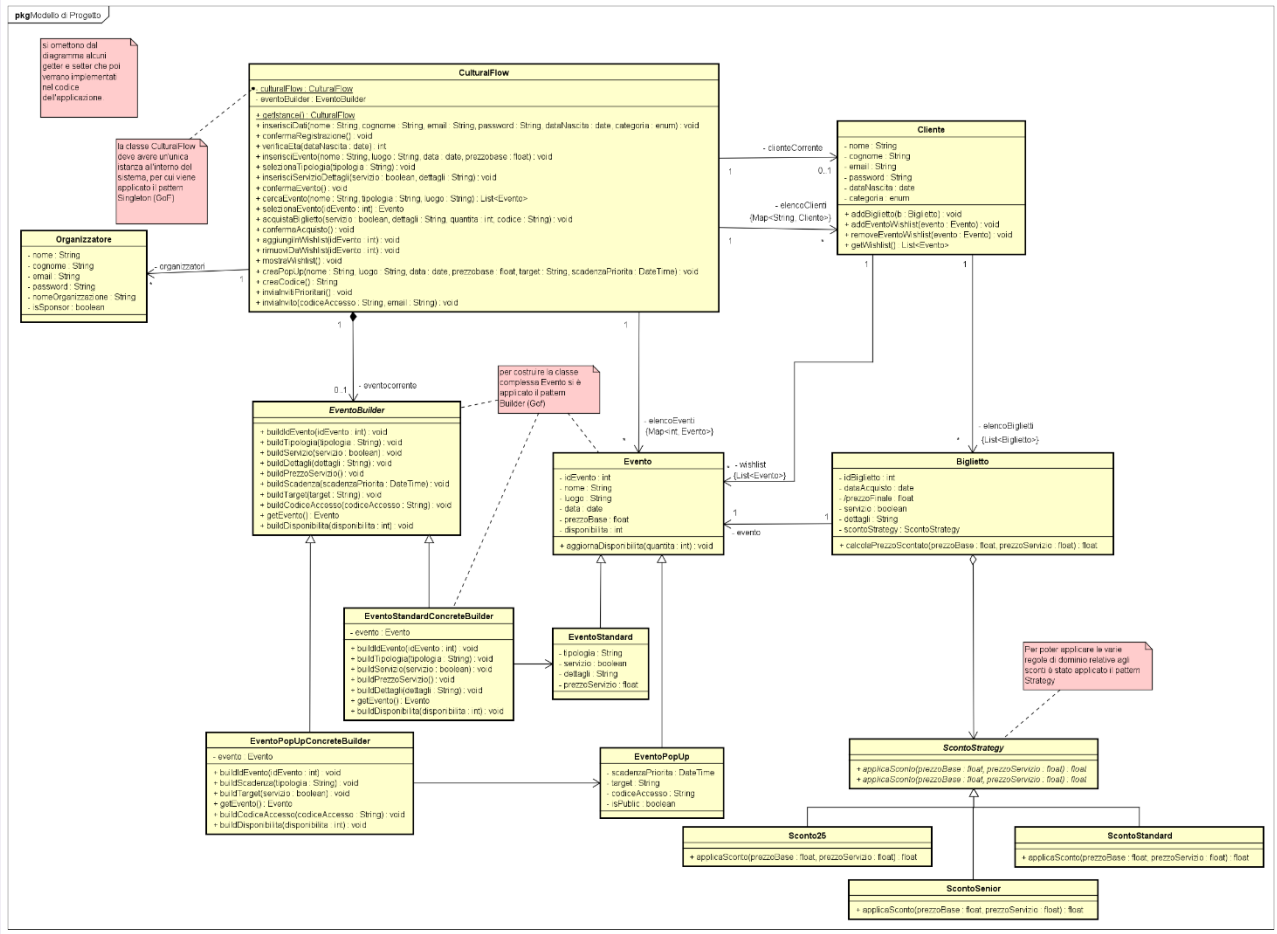


3.4.2. inviaInvitiPrioritari()



3.4. Diagramma delle Classi (DCD)

Nel seguente diagramma delle classi si può notare l'integrazione delle nuove entità individuate in questa iterazione. La struttura del pattern Builder è stata estesa per supportare la creazione di un nuovo tipo di Evento. È stato, inoltre, introdotto il pattern Strategy con le relative classi concrete per la gestione delle logiche di calcolo del prezzo e degli sconti.



4. Testing

Durante la seconda iterazione, alcuni metodi di testing definiti nella prima iterazione sono stati rivisti e modificati per adattarsi ai cambiamenti introdotti nel sistema durante questa fase. Inoltre, sono stati testati i nuovi metodi implementati per coprire i casi d'uso della seconda iterazione.

Riportiamo di seguito i nuovi metodi di test:

CulturalFlowTest

1. testCreaPopUp()

Verifica che il processo di creazione di un Evento Pop Up funzioni correttamente, assicurandosi che l'Organizzatore risulti essere uno Sponsor.

2. testInviaInvitiPrioritari()

Verifica la regola di dominio R8, inviando gli inviti a coloro che hanno nella propria wishlist almeno tre Eventi con tipologia uguale al target inserito.

3. testInserisciEventoStandard()

Verifica che l'inizio del processo di creazione di un Evento funzioni, assicurandosi, inoltre, che venga impostato correttamente come evento corrente. (Modifica del test *testInserisciEvento()*, Iterazione 1).

4. testSelezionaEvento()
Verifica che il sistema sia in grado di recuperare correttamente un Evento specifico dall'elenco degli Eventi tramite il suo id.

5. testAcquistaBiglietto()
Verifica che l'acquisto di un Biglietto vada a buon fine, analizzando le varie regole di dominio e l'applicazione dei relativi sconti.

6. testConfermaAcquisto()
Verifica che la conferma dell'acquisto di un Biglietto funzioni e che il Biglietto venga inserito nell'elenco dei biglietti acquistati dal Cliente.

7. testAggiungiInWishlist()
Verifica che l'inserimento di un nuovo Evento all'interno della wishlist di un Cliente venga effettuato correttamente.

8. rimuoviDaWishlist()
Verifica che la rimozione di un nuovo Evento all'interno della wishlist di un Cliente venga effettuata correttamente.

9. testMostraWishlist()
Verifica che il sistema sia in grado di restituire correttamente l'elenco completo degli eventi che un utente ha salvato come preferiti.

BigliettoTest

1. testCalcoloPrezzoScontato()
Verifica che lo sconto sull'acquisto di un Biglietto venga applicato correttamente.

ClienteTest

1. testAddEventoWishlist()
Verifica che un Evento venga correttamente inserito all'interno della Wishlist del Cliente corrente.

2. testHaInteresseElevato()
Verifica la regola di dominio R8, accertandosi che il Cliente abbia nella propria Wishlist almeno tre Eventi della stessa tipologia del target.

EventoPopUpTest

1. testIsAcquistabile()
Verifica che la possibilità di acquistare un Biglietto, nello specifico verifica se c'è disponibilità e se la data attuale è precedente a quella di scadenzaPriorita (e quindi l'acquisto è permesso solo con il codice corretto). Se la data di scadenza è passata, l'Evento deve risultare acquistabile per chiunque.

EventoStandardTest

1. testIsAcquistabile()
Verifica che la possibilità di acquistare un Biglietto, nello specifico verifica la disponibilità.

EventoTest

1. testIsAcquistabile()
Verifica che la possibilità di acquistare un Biglietto.

2. testAggiornaDisponibilita()
Verifica che la quantità di posti disponibili per un evento venga decrementata correttamente in seguito a un acquisto.

ScontoStandardTest

1. testApplicaScontoStandard()
Verifica che la strategia di prezzo predefinita del sistema calcoli correttamente il totale senza applicare alcuna riduzione di prezzo.

Sconto25Test

1. testApplicaSconto25()
Verifica che la strategia di sconto del 25% calcoli correttamente la riduzione di prezzo sul totale.

ScontoSeniorTest

1. testApplicaScontoSenior()
Verifica la specifica logica di calcolo riservata ai clienti appartenenti alla categoria Senior.